Guiding Generative Probabilistic Weather Models to Simulate Realistic Extreme Weather Events

Master Thesis - Notes

Author

Alain Joss `alain.joss@uzh.ch`

Supervision

Maxim Samarin `maxim.samarin@sdsc.ethz.ch`
Shirin Goshtasbpour `shirin.goshtasbpour@sdsc.ethz.ch`
Prof. Dr. Guillaume Obozinski `guillaume.obozinski@sdsc.ethz.ch`

May 12, 2026

1 ArchesWeather

Here a few clarifying notes on the specifics of ArchesWeather and its implementation details in code.

1.1 Empirical observations

- The diffusion model on top of the deterministic one makes the predictions worse in terms of RMSE.
- The predictions seem to be largely a copy paste of the past.

1.2 The theory

Instead of regressing the learned flow model against the velocity vector $r - \epsilon$, they learn to predict r directly, and retrieve the direction $u_t^\theta(r_t)$ for the next update as follows:

$$\begin{aligned}
& z_t = (1-s)r_t + s\epsilon \\
\text{let} \quad & r_t^\theta := r_t^\theta(z_t) = r_t^\theta((1-s)r_t + s\epsilon) \\
\text{s.t.} \quad & \epsilon_t^\theta = \frac{z_t - (1-s)r_t^\theta}{s} \\
\text{then} \quad & u_t^\theta = r_t^\theta - \epsilon_t^\theta \\
& \quad = \frac{s}{s} r_t^\theta - \frac{z_t - (1-s)r_t^\theta}{s} \\
& \quad = \frac{r_t^\theta - z_t}{s}
\end{aligned}$$

Note that our network directly predicts the denoised residual r_t^θ, which is a clean prediction of the state at time t, different from z_{t+1}, which is only gradually being denoised. To make the Euler update we go back to the noisy space by computing the velocity vector u_t^θ, which is the scaled difference between the last noisy state and the current one. This could be viewed as a gradual "cleansing" step. The scaling term $s = \frac{t}{T}$, progressing from 1 to 0, and enables to do "more" update at the beginning of the sampling procedure, and gradually less. Given the velocity vector $u_t^\theta(z_t)$, we then do the Euler update as follows:

$$z_{t+\delta} = z_t + h u_t^\theta(z_t)$$

1.3 The implementation

For some reason, although the sampler performs really simple updates, they defer the update steps to the `diffusers.FlowMatchEulerDiscreteScheduler` class, which overcomplicates thing unnecessarily.

Since the scheduler makes use of $h := s_t - s_{t+1} < 0$, they compute the negative velocity $-u_t^\theta = \frac{r_t^\theta - z_t}{s}$.

The scheduler updates as outlined in algorithm 1.

Since with our parameters $\hat{s}_t = s_t$, $\Delta t = -u_t$ and $d = s_t - s_{t+1}$, making the update equivalent to the simple Euler sampler defined above.

Algorithm 1 Update

Require: current sample z_t, model output $-u_t$, noise level s_t, effective noise level $\hat{s}_t$, next scheduler value s_{t+1}

$\tilde{z} \leftarrow z_t - u_t \cdot s_t$ ▷ compute denoised sample

$d \leftarrow \dfrac{z_t - \tilde{z}}{\hat{s}_t}$ ▷ convert denoised prediction into ODE derivative

$\Delta t \leftarrow s_{t+1} - \hat{s}_t$ ▷ step size

$z_{t+1} \leftarrow z_t + d \cdot \Delta t$ ▷ Euler update

return x_{t+1}

2 Guidance @ flow time

2.1 The guidance term

For a weather-time step n, we define the guidance target y_n as a scalar representing the average value of a specific variable over a selected 2D region. Importantly, we define y_n in denormalized space, that is, in the original data space.

2.2 Mask

The selected region together with the variable of interest defines a binary mask $\mathcal{M}$. Importantly, the mask has the same shape as a weather state: it is zero everywhere except on the entries corresponding to the chosen variable and spatial region.

2.3 The mask term

At flow time t, given the mask $\mathcal{M}$ and the current clean prediction $\hat{x}_t$, we define the mask term $m(z_t)$ as the average of the predicted values over the mask:

$$m(z_t) = \frac{\sum_i \mathcal{M}_i \, \hat{x}_t^i}{\sum_i \mathcal{M}_i}.$$

To compute $\hat{x}_t$, we need to undo the normalizations used in the model.

First, we map the predicted residual in flow space, $r_t^\theta(z_t)$, back to normalized data space:

$$\hat{x}_t^{\text{norm}} = \hat{x}^{\text{det}} + \sigma^{\text{res}} \odot r_t^\theta(z_t),$$

where $\hat{x}^{\text{det}}$ is the prediction of the deterministic model, and σ^{res} is the residual scaling constant used during training, with the spatial dimensions =1 and same number of levels and variables as r_t^θ. Note that we are correctly using the clean prediction of the residual r_t^θ, as opposed to the noisy state z_{t+1}.

We then map this normalized prediction back to the original data space using the inverse z-normalization:

$$\hat{x}_t^{\text{denorm}} = \bar{x}^{\text{data}} + \sigma^{\text{data}} \odot \hat{x}_t^{\text{norm}},$$

where $\bar{x}^{\text{data}}$ and σ^{data} are the mean and standard deviation of the training data.

Putting everything together, we obtain

$$\begin{aligned} m(z_t) &= \frac{\sum_i \mathcal{M}_i \, [\hat{x}_t^{\text{denorm}}]^i}{\sum_i \mathcal{M}_i} \\ &= \frac{\sum_i \mathcal{M}_i \, [\bar{x}^{\text{data}} + \sigma^{\text{data}} \hat{x}_t^{\text{norm}}]^i}{\sum_i \mathcal{M}_i} \\ &= \frac{\sum_i \mathcal{M}_i \, [\bar{x}^{\text{data}} + \sigma^{\text{data}} (\hat{x}^{\text{det}} + \sigma^{\text{res}} r_t^\theta(z_t))]^i}{\sum_i \mathcal{M}_i}. \end{aligned}$$

The reason for going back to denormalized data space is that the guidance target y_n is naturally defined there. In contrast, defining the corresponding target directly in normalized space is not straightforward, because the normalization depends on variable-wise training statistics and on the residual parameterization.

2.4 Loss

Now that we have both the mask term $m(z_t)$ and the guidance target y_n, we define the loss of interest as the squared ℓ_2 distance:

$$\begin{aligned}\mathcal{L}(y_n, z_t) &= \frac{1}{2}\left\|y_n - m(z_t)\right\|^2 \\ &= \frac{1}{2}\left\|y_n - \frac{\sum_i \mathcal{M}_i \left[\bar{x}^{\text{data}} + \sigma^{\text{data}}(\hat{x}^{\text{det}} + \sigma^{\text{res}} r_t^\theta(z_t))\right]^i}{\sum_i \mathcal{M}_i}\right\|^2 .\end{aligned}$$

We are then interested in the gradient $\nabla_{z_t}\mathcal{L}$, which we compute using automatic differentiation.

3 Guidance methods

3.1 Classifier guidance

The conditional vector field can be written in terms of the conditional score as

$$u_t(z_t \mid y) = u_t(z_t) + \alpha_t \nabla_{z_t} \log p_t(y \mid z_t),$$

where α_t is a time-dependent scaling term determined by the noise schedule.

We are free to choose a convenient form for $p_t(y \mid z_t)$, and we define it through the loss:

$$p_t(y \mid z_t) \propto e^{-\mathcal{L}(y, z_t)}.$$

This gives

$$\begin{aligned} u_t(z_t \mid y) &= u_t(z_t) + \alpha_t \nabla_{z_t} \log p_t(y \mid z_t) \\ &= u_t(z_t) + \alpha_t \nabla_{z_t} \log e^{-\mathcal{L}(y, z_t)} \\ &= u_t(z_t) - \alpha_t \nabla_{z_t} \mathcal{L}(y, z_t). \end{aligned}$$

In the spirit of classifier guidance, we replace α_t by a more general guidance-strength term

$$\lambda_t := w \cdot \alpha_t,$$

which leads to the guided vector field

$$\tilde{u}_t(z_t \mid y) = u_t(z_t) - \lambda_t \nabla_{z_t} \mathcal{L}(y, z_t).$$

Note that for $w \neq 1$, we no longer recover the original conditional vector field $u_t(z_t \mid y)$ exactly, but instead use the modified field $\tilde{u}_t(z_t \mid y)$. As in standard classifier guidance, this tuning improves controllability at the cost of the original probabilistic interpretation.

The schedule λ_t is therefore a parameter to tune for our purposes. This tuning requires a notion of fitness, that is, a metric describing how good the guided samples look.

In our setting, we are not interested in the classifier-free guidance counterpart, since we do not train a conditional network $u_t(z_t \mid y)$. Still, for completeness, one could experiment with convex combinations of the base velocity and the guidance term, in the spirit of classifier-free guidance:

$$\bar{u}_t(z_t \mid y) := (1 - w) u_t(z_t) - w \lambda_t \nabla_{z_t} \mathcal{L}(y, z_t),$$

or even with a time-dependent weight,

$$\bar{u}_t(z_t \mid y) := (1 - w_t) u_t(z_t) - w_t \nabla_{z_t} \mathcal{L}(y, z_t).$$

These variants are heuristic and do not follow directly from a probabilistic derivation.

3.2 Monte Carlo classifier guidance

Following the Monte Carlo refinement of loss-guided diffusion, we replace the single-point guidance term by a local sampling-based estimate around z_t. We introduce a surrogate Gaussian distribution centered at the current point:

$$q_t(\tilde{z}_t \mid z_t) := \mathcal{N}(z_t, r_t^2 I),$$

and draw i.i.d. samples

$$\tilde{z}_t^j \sim q_t(\cdot \mid z_t), \qquad j = 1, \dots, G.$$

In practice, we use the reparameterization

$$\tilde{z}_t^j = z_t + r_t \varepsilon_j, \qquad \varepsilon_j \sim \mathcal{N}(0, I),$$

so that each sampled point remains a differentiable function of z_t.

Instead of averaging the loss gradients directly, we define the Monte Carlo guidance term as

$$g_t^{\mathrm{MC}}(z_t, y) := \nabla_{z_t} \log \left(\frac{1}{G} \sum_{j=1}^{G} \exp\Big(-\mathcal{L}(y, \tilde{z}_t^j)\Big) \right).$$

Differentiating this expression shows that the update is a weighted combination of local loss gradients:

$$g_t^{\mathrm{MC}}(z_t, y) = -\sum_{j=1}^{G} w_j \nabla_{\tilde{z}_t^j} \mathcal{L}(y, \tilde{z}_t^j),$$

where

$$w_j := \frac{\exp\Big(-\mathcal{L}(y, \tilde{z}_t^j)\Big)}{\sum_{k=1}^{G} \exp\Big(-\mathcal{L}(y, \tilde{z}_t^k)\Big)}.$$

Hence, samples with smaller loss receive larger weight, while samples with larger loss are downweighted. This is the key difference from plain gradient averaging.

The guided vector field then becomes

$$\tilde{u}_t(z_t \mid y) = u_t(z_t) + \lambda_t \, g_t^{\mathrm{MC}}(z_t, y).$$

Compared with standard classifier guidance, this replaces the single local guidance direction by a locally averaged, loss-weighted one. Standard pointwise guidance is recovered in the limiting case of a single sample together with vanishing perturbation radius.

3.3 Gradient-averaged local guidance

As a simpler alternative, we can average the loss gradients evaluated at perturbed points around z_t. Using the same proposal distribution

$$q_t(\tilde{z}_t \mid z_t) := \mathcal{N}(z_t, r_t^2 I),$$

we draw i.i.d. samples

$$\tilde{z}_t^j = z_t + r_t \varepsilon_j, \qquad \varepsilon_j \sim \mathcal{N}(0, I), \qquad j = 1, \ldots, G.$$

We then compute the average gradient

$$\bar{g}_t(z_t, y) := \frac{1}{G} \sum_{j=1}^{G} \nabla_{\tilde{z}_t^j} \mathcal{L}(y, \tilde{z}_t^j),$$

and use it to define the heuristic guided vector field

$$\tilde{u}_t(z_t \mid y) = u_t(z_t) - \lambda_t \bar{g}_t(z_t, y).$$

Unlike the Monte Carlo guidance term above, this update is not derived from the loss-guided diffusion formulation. Instead, it should be viewed as a practical smoothing heuristic: all sampled gradients contribute equally, regardless of their loss value.

3.4 FlowGrad

An alternative to hand-designing a schedule for λ_t is to define an optimization problem that determines suitable guidance strengths automatically:

...

4 Guidance @ weather time

In the previous section we took the guidance term y_n at time n as a given. We now want to answer the question of how to specify a trajectory $\{y_n\}_{n=1}^N$.

4.1 M-Rollout trajectory

A key feature we were interested in is that the both the single sampled weather states and the weather trajectory are phisically plausible although extreme.

A simple heuristic to produce an unguided rollout ensemble with M models over N steps, and capture the distribution of $m(\hat{x}_n^m)$ over time. We can then define a target trajectory of scalar guidance terms, and steer the M models towards it in a guided rollout.

Out of the box we can define 3 guidance modes, where we respectively guide towards:

- a percentage change from the mean
- lower bound
- upper bound

of the unguided rollout.

To test what guidance is doing in the first place we could also guide towards the ground truth trajectory, and assess where and how different the guided trajectories are wrt to the ground truth.

4.2 Experimental setting

4.3 Summary of hyperparameters

In the following we summarize all hyperparameters for generating a guided trajectory:

- x_0: start state
- N: length of the trajectory in weather time
- M: number of ensemble members for the guidance base distribution
- λ_t: guidance strength schedule
- G: number of montecarlo drawings
- $\mathcal{M}$: mask (variable at level and region of interest)
- $\mathcal{K}$: mask kernel defining the weigths for the averaging over the region
- ρ_n: trajectory shape (relative to M rollout)

From this table, the hyperparameter that most importantly has to be ablated for tuning and understanding is λ_t. In fact, it's really not straight forward to predict its influence on the guided states and the realized guidance.

4.4 Evaluation

The evaluation of the experiments straight forawrdly includes the realized guidance as key metric. In addition, we want to track the development of other variables. This can include the sum of relative change over the whole variable-level slice and the distributions of values.

A plausible method of evaluation is to guide sampling towards the ground truth and evaluate whether it improves the accuracy of other variables (not the guided one). This might gives an idea whether the produced states are simply some adversarial noise, or the gradient direction is qualitatively good.

5 Results

5.1 Mask driven guidance

- The difference across

6 Old notes - Generating weather trajectories

This section is devoted towards defining different heuristics to specificy guidance trajectory over N step rollouts.

6.1 Guidance times

Since the goal is to generate a weather trajectory, which is defined by a roll-out of multiple prediction steps, we have to distinguish between guidance at universal time and guidance at diffusion time.

6.1.1 Guidance @ universal time

Depending on the type of model used we can have different types of guidance:

- Deterministic: backwards guidance
- Generative: forward guidance
- Deterministic + residual generative: rolling forward/backward guidance

6.1.2 Guidance @ generation time

At generation time we have a single r^{guide} that define the direction of generation. A guidance strength schedule defines how strong we want to guide the generation towards the wanted result over T steps.

The guidance method used depends in the first place on the type of model we are dealing with:

- Score: $\nabla_{x_t} \log p_t(x_t|c)$
  - Loss guidance
  - Universal guidance
- Flow matching: $u_t(x_t|c)$
  - FlowGrad
  - CFG-like
  - Source-Guided Flow Matching
  - Monte Carlo guidance

Importantly, for score based models use

6.2 Trajectory hyperparameters

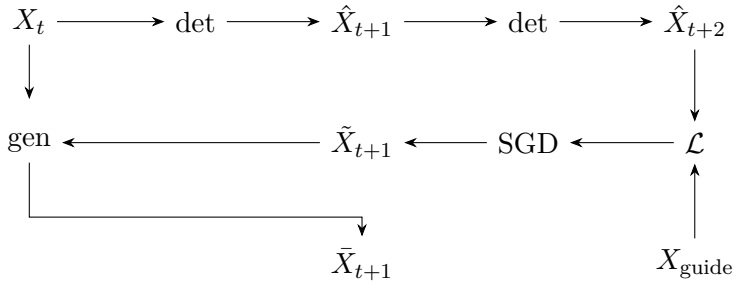
The design of r^{guide} depends on our approach to specify the weather trajectory. To simplify matters we can start by listing relevant hyperparameters.

6.3 Backward guidance

6.4 Forward guidance

6.5 Rolling forward/backward guidance

Must pay attention to the fact that the input states are actually 2! In the gradient descent we should only change the current state!



7 Old notes on universal guidance

7.1 UG step 2

Before analyzing their second adjustment to the guidance function, we have to answer following question: what is the loss and its components in our case? In our case we're interested in following loss

$$\mathcal{L}(c = (\mathbb{M}, r_{\text{guide}}), r^{\theta}(r_t)) = \|\text{avg}(\mathbb{M} \odot (r^{\text{guide}} - r^{\theta}(r_t)))\|_2^2$$

Note that in our case $f := \text{Id}(\cdot)$ (identity function). In fact we want to compute the loss with respect to the predicted clean sample, without passing it into any additional function. Here $\mathbb{M}$ denotes a mask of interest and r^{guide} contains the corresponding values of interest. For instance r^{guide} might just be a percentage increase in temperature at a specific location.

In our case, the second adjustment of the guidance function results in following

$$\begin{aligned}
\Delta_{z_o} &= \arg\min_{\Delta} \mathcal{L}(r^{\text{guide}}, z^{\theta} + \Delta) \\
\Delta_{z_o} &= z^{\theta} - r^{\text{guide}} \\
\tilde{z} &= z^{\theta} - \Delta_{z_o} = r^{\text{guide}}
\end{aligned}$$

Because the optimized term can be computed in closed form (due to the $f := \text{Id}(\cdot)$), the update $\tilde{z}$ collapses to r_{guide}. This is of course not what we want, since r_{guide} is not physically meaningful, and the procedure itself is just a hack to make guidance more reliable than the first version.